Exercise: **Set up Remote Access**

Setting Up Remote Access from Kali Linux to Xubuntu

In this scenario, my username on Xubuntu is student, and its IP address is 172.30.1.20.

I already generated an Ed25519 SSH key pair on Kali Linux. I also know the password for the student account on Xubuntu.

My goal is to set up SSH key-based authentication so I can log in without entering the student's account password.

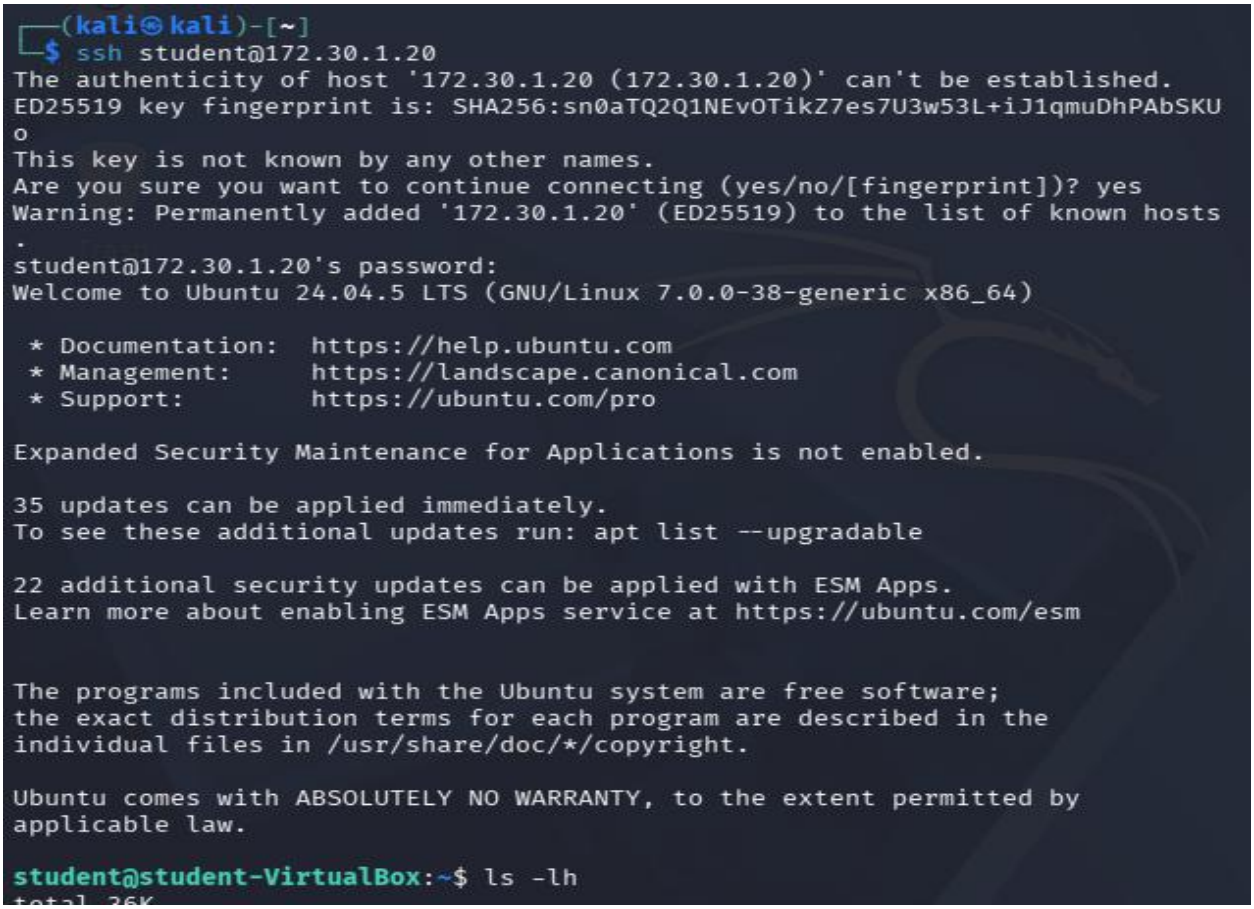
This is what I'm going to do:

Step 1: On my Kali machine, I use SSH to log in to Xubuntu with the following command:

ssh <username>@<ubuntu_ipaddr>

example: ssh student@172.30.1.20

the Terminal will ask for student's password so I just typed it in: ******



After logging into the student account on Xubuntu through SSH from Kali, I will create a .ssh directory in the student's home directory.

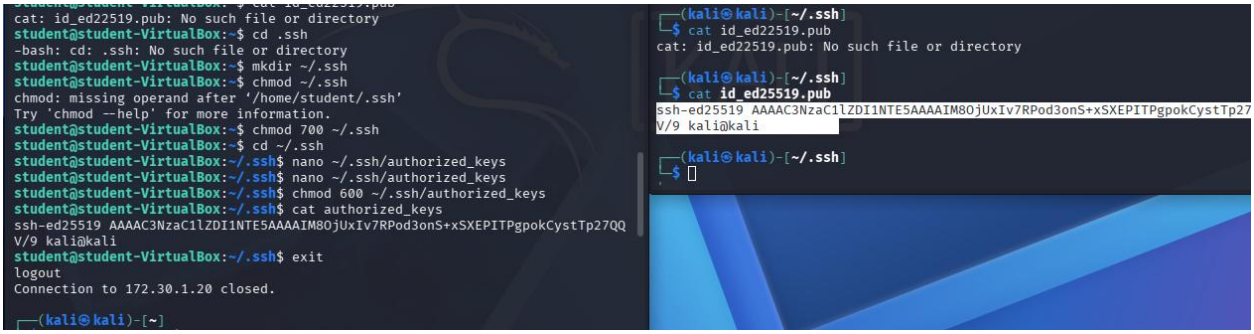
Inside this directory, I will create a file called authorized_keys to store Kali's public SSH key.

```
student@student-VirtualBox:~$ mkdir ~/.ssh
student@student-VirtualBox:~$ chmod ~/.ssh
chmod: missing operand after '/home/student/.ssh'
Try 'chmod --help' for more information.
student@student-VirtualBox:~$ chmod 700 ~/.ssh
student@student-VirtualBox:~$ cd ~/.ssh
student@student-VirtualBox:~/.ssh$ nano ~/.ssh/authorized_keys
student@student-VirtualBox:~/.ssh$ nano ~/.ssh/authorized_keys
student@student-VirtualBox:~/.ssh$ chmod 600 ~/.ssh/authorized_keys
student@student-VirtualBox:~/.ssh$ cat authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIM8OjUxIv7RPod3onS+xSXEPITPgpokCystTp27QQ
V/9 kali@kali
student@student-VirtualBox:~/.ssh$ exit
```

I used nano to open the authorized_keys file on Xubuntu. Then, I copied the public key from another terminal on Kali and pasted it into the file.

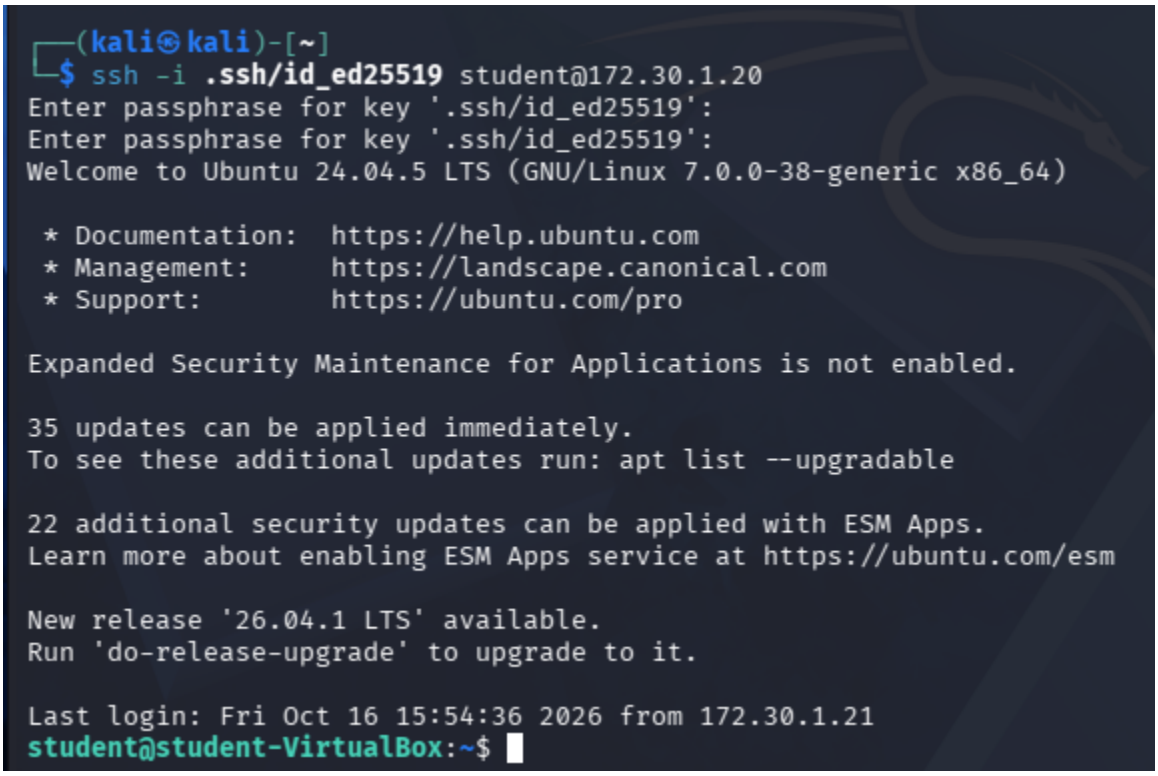
After saving the file, I set the appropriate permissions and tested the SSH connection using my private key.

All commands are entered from Kali Linux, but commands executed inside the SSH session run on Xubuntu.



At this point, I no longer need the student's account password because Xubuntu now trusts my public key. I can authenticate using the matching private key stored on Kali:

ssh -i .ssh/id_ed25519 student@172.30.1.20



Now that remote access is working, I will demonstrate four ways to secure the SSH server.

-Disable Password Authentication (no one can remote login with username and password – including hacker)

-Changing SSHD default port (security through obscurity – hacker will take more time than usual to find your main port to attack)

-Disable Root Login (Root can't remote log in – prevent hacker access with full privilege)

-Limit Remote Users(Choose specific users to remote access)

Exercise: Disable password authentication

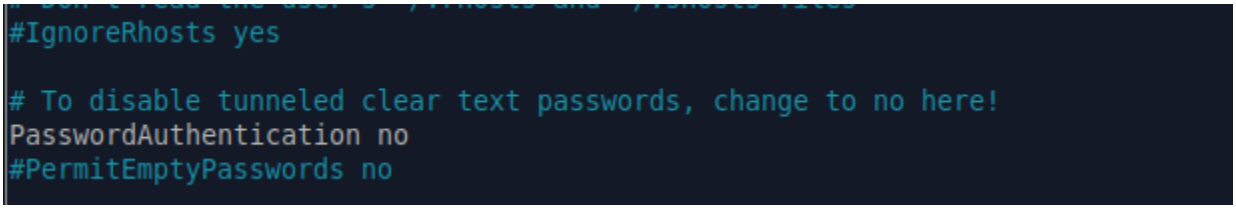
Why do we need to do this exercise? By disabling password authentication, we remove one method of logging into the SSH server. Users must authenticate using another permitted method, such as SSH public key authentication.

This reduces the risk of password-based attacks, including brute-force attacks and password guessing.

Step 1: On xubuntu machine, we go to edit the file /etc/ssh/sshd_config by using the command:

sudo nano /etc/ssh/sshd_config

Step 2: Inside the file, search for the line “PasswordAuthentication” , delete the “#” and set it to “no”.

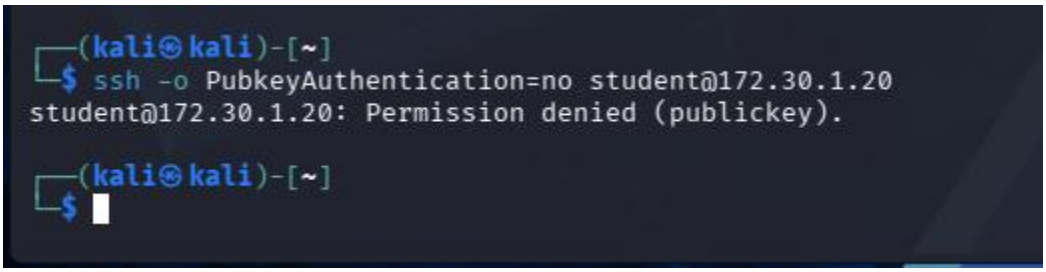


Ctrl O + enter to save the file then exit the file.

Step 3: Restart SSHD server by using the command:

sudo systemctl restart ssh

To test this configuration, go to Kali Linux and attempt to log in using password authentication only. The login attempt should fail.



Troubleshooting: If the login still succeeds, the SSH client may be automatically using my private key stored on Kali.

To test password authentication without using public key authentication, I can run:

ssh -o PubkeyAuthentication=no student@172.30.1.20

Since password authentication is disabled on Xubuntu, the login attempt should fail.

Note: The SSH client automatically attempts public key authentication when a suitable private key is available. Therefore, disabling password authentication on the server does not prevent successful key-based logins.

Exercise: Changing SSHD default port

Why do we need to do this exercise? Many automated bots scan the Internet for SSH servers running on the default port 22. By changing the SSH port to a less common port, such as 4444, we may reduce the number of automated login attempts targeting the default port. However, this is not a strong security measure because attackers can still discover the new port through port scanning.

This technique is commonly known as *security through obscurity*.

Step 1: Open /etc/ssh/sshd_config by sudo nano /etc/ssh/sshd_config.

Step 2: Search for “Port 22” and set it to “Port 4444” -> This is the less common port mentioned earlier.



Then ctrl O + enter to save and ctrl X to exit

Step 3:Restart SSHD server by using the command(Xubuntu 24.04):

sudo systemctl daemon-reload

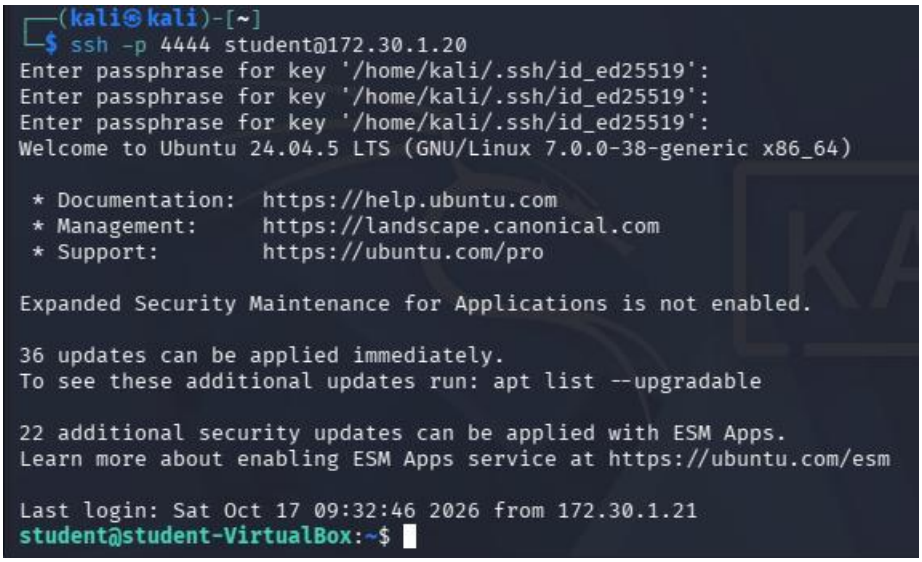
sudo systemctl restart ssh.socket

That’s it! You can test it by login through the new port on Kali (test vm) by:

ssh -p 4444 username@ipaddr

For example if I want to remote login into student account at ip 172.30.1.20:

Ssh -p 4444 student@172.30.1.20



Exercise: Disable root Login

Why do we need to do this exercise? We all know that the root account has the highest privileges on a Linux system, right?

Now imagine an attacker successfully logging into your server as root. They could modify system configurations, delete important files, or compromise the entire system. By disabling direct root login through SSH, we reduce this risk. Administrators can still log in using their regular accounts and use sudo when administrative privileges are required.

Step 1: Open /etc/ssh/sshd_config by:

sudo nano /etc/ssh/sshd_config.

Step 2: Search for “PermitRootLogin” and set it to “no”

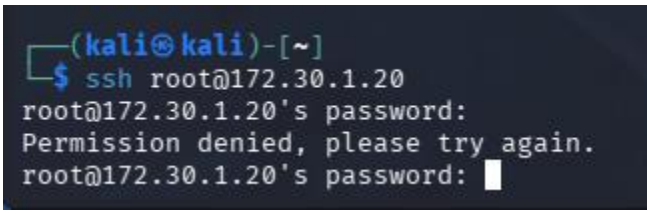


Step 3: Restart SSHD server by:

sudo systemctl restart ssh

That's it! You can test it by remote login as root to your xubuntu from kali by this command (should fail):

Ssh root@172.30.1.20



Exercise: Limit Remote Users

Why do we need to do this exercise?

As system administrators, we should not allow every user to access the server remotely through SSH. Instead, we should only allow specific users who need remote access to perform their tasks.

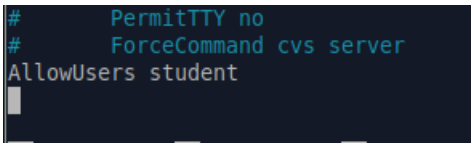
This follows the principle of least privilege and reduces the number of accounts that attackers can target through SSH.

Step 1: Open /etc/ssh/sshd_config by:

sudo nano /etc/ssh/sshd_config.

Step 2: Add the AllowUsers directive followed by the username:

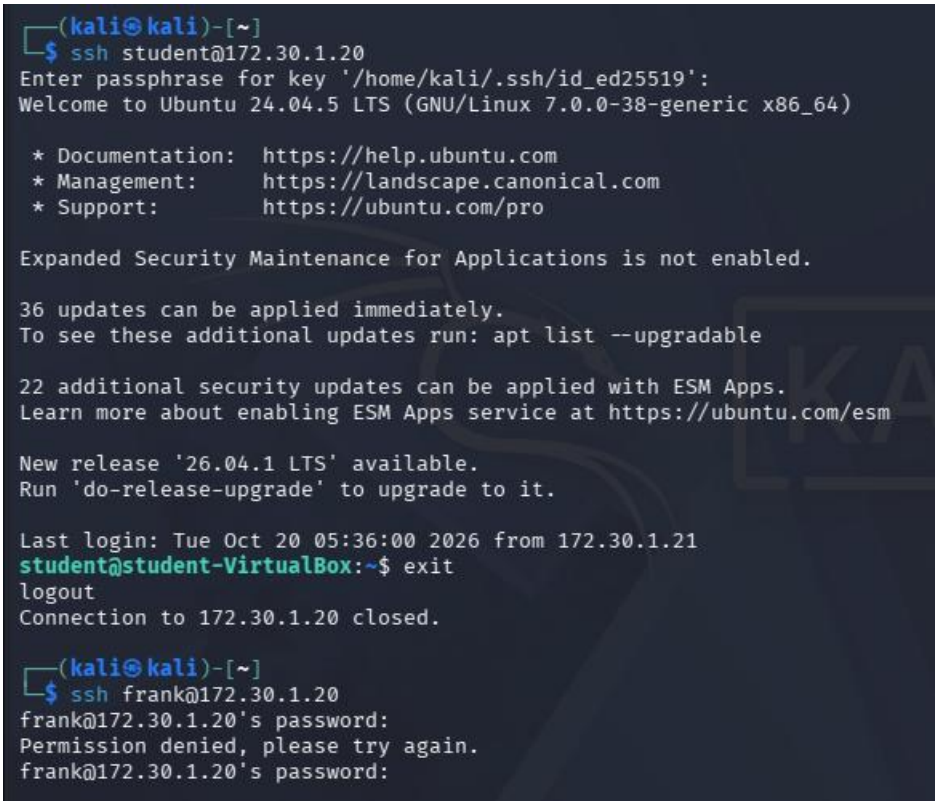
Example: AllowUsers student



Step 3: Restart SSHD server:

sudo systemctl restart ssh

That's it! To verify the configuration, I attempted to log in using two different accounts: student and frank. The student account successfully connected to the SSH server, while the frank account was denied access:



-This confirms that only the user specified in the AllowUsers directive is permitted to log in through SSH.

Note: Before restarting the SSH server, I use **sudo sshd -t** to check the configuration for syntax errors.